Automation

This document is the technical policy and guideline for how a developer writes and organizes automations that manage systems.

Purpose

This standard tells a developer how to structure service-management automations, where to store desired-state representations, and how to keep one-time work out of Git-managed playbooks. It applies to any automation platform (for example Ansible, Terraform, or Chef). The audience is automation developers and operators who run those automations.

Definitions

An *automation* is a script, playbook, module, recipe, or program used to manage an environment of interacting systems. A system may be an application or an operating system on a hardware system.

An *automation developer* is a person or AI that implements an automation. After this definition, this document uses *developer* to mean automation developer.

An *operator* is a person or AI that runs automations, but does not have the privileges to develop production automations.

A *playbook* is a named automation unit, regardless of the automation platform.

An *automation directory* is the directory that holds the playbooks (or equivalent units) for a service or subsystem. This document also calls that directory a *module*.

A *service-management playbook* is a top-level playbook in an automation directory. It performs one service-management verb and is how an operator manages that module in a consistent way. Its filename is the verb only (`deploy.yml`, `start.yml`, and the other names in the service-management table).

A *task playbook* is a playbook that one or more service-management playbooks call to implement a specific or complex function. A task playbook is not itself a service-management verb.

A *common directory* is an optional subdirectory of an automation directory that holds variables shared by more than one playbook in that module.

A *representation* is a document that states the desired configuration of an object on a managed system, in a form another automation tool or a manual client can apply. Typical forms are JSON, YAML, or source text. A representation is not an orchestration step (lookup, create-versus-update, credentials, or gathered facts).

A *commentary file* is a Markdown file with the same basename as a representation file. It holds notes that do not belong in the representation itself.

An *ephemeral automation* is a one-time play used to create, delete, rename, or deactivate an obsolete configuration, or to perform another migration that will not remain part of the desired state.

A *transient automation* is an automation created for a focused diagnostic or repair. Transient automations are typically short-lived scripts.

Terraform is a declarative infrastructure-as-code automation platform. A developer declares the desired resources in a configuration; Terraform computes the difference between that configuration and its *state*, then applies the difference to the managed system.

A *resource* is the smallest object a Terraform configuration can declare and manage as one unit. A provider may represent an entire collection as a single resource, with no resource type for one member of that collection.

State is the record Terraform keeps matching each resource in a configuration to the corresponding real-world object and its last known attributes. Terraform computes a plan by comparing a configuration against this record, refreshed against the real objects.

A *backend* is where Terraform stores and retrieves state. A *local backend* keeps state in a file readable only on the machine running Terraform. A *shared backend* (or *shared state*) keeps state in a location more than one developer, operator, or automation run can read and write.

State locking is a capability of a shared backend that prevents two concurrent Terraform operations against the same state from running at once.

The service-management playbook names and their meanings are:

Playbook name	Description
build	Aggregate any artifacts needed to build a service and then compile or transform the artifacts into a form suitable to deploy or install the service
deploy	Deploys all the resources to a host or container that runs the service
check	Compares the deployed resources against their representation files and reports what would change, without changing anything

Playbook name	Description
install	Marshals all the resources needed for a service onto a control node, a managed node, or a local repository
start	Starts the service on the container or host without having to reinstall or redeploy any resources
diagnose	Observes and reports current health and state of the service and prerequisites with read-only checks. The name **status** is obsolete; a developer must use **diagnose**.
test	Asserts expected behavior by exercising workflows end-to-end and returning pass/fail results. May create short-lived test artifacts and should clean them up.
stop	Stops the service running
undeploy	Removes all the resources needed for a service from a host or container
uninstall	Removes all transient resources from the control node with the exception of any source code from a repository

Verb semantics

build

- Purpose: transform source or templates into deployable artifacts.
- Typical behavior: compile, package, render, stage.
- Side effects: artifact outputs only.

install

- Purpose: prepare hosts and control nodes with prerequisites.
- Typical behavior: install packages, users, directories, runtimes, base configs.
- Difference vs deploy: install prepares substrate; deploy places service resources on that substrate.
- Side effects: host-level state changes expected and persistent.

deploy

- Purpose: place and configure service-specific resources so the service can be started.
- Typical behavior: apply manifests, distribute configs, create service accounts, mount points.
- Difference vs install: deploy targets service composition; install targets foundational prerequisites.
- Side effects: service artifacts and manifests present, but the service may still be stopped.

check

- Purpose: answer "does the deployed configuration match the representation files?"
- Typical behavior: dry-run the deploy playbook's resource application (for example, a diff-only sync, or the automation platform's own plan/dry-run mode) and report what would change.
- Difference vs diagnose: check compares deployed state to the representation files (configuration drift); diagnose compares live state to expected service health (operational status). Check does not assert the service is healthy; diagnose does not compare against representation files.
- Side effects: none. Read-only, safe to run at any time after deploy.

start

- Purpose: transition a deployed service to running.
- Typical behavior: start daemons, scale workloads up, enable runtime routes.
- Side effects: runtime activity starts.

diagnose

- Purpose: answer "what is the current state and what is wrong?"
- Behavior characteristics:
- Read-only checks where possible.
- Safe to run repeatedly in broken or healthy environments.
- Reports facts (pods, ports, endpoints, cert dates, mounts, logs).
- Must include all checks in the corresponding test playbook for the same service.
- Must be a superset of the corresponding test playbook by adding deeper diagnostics, richer evidence, and remediation hints.
- May run additional in-depth probes that are heavier than test.
- Preferred outputs:
- Structured table or list of checks with status and details.
- Clear remediation hints for each failed check.

test

- Purpose: answer "does the service behavior meet expectations?"
- Behavior characteristics:
- Executes workflow assertions (for example write to read, submit to complete, emit to observe).
- Produces explicit pass/fail outcomes suitable for CI or release gates.
- May create temporary test artifacts and should clean them up.
- Should fail fast with actionable diagnostics.
- Relationship to diagnose:
- Test is the baseline check set for service correctness.
- Diagnose includes all test checks plus deeper diagnostic checks.

- A test that wholly depends on another test must not run when the prerequisite test fails.

stop

- Purpose: transition a running service to a non-running state without removing deployment artifacts.
- Typical behavior: stop daemons, scale workloads down.

undeploy

- Purpose: remove deployed service resources from targets.
- Typical behavior: delete manifests or resources, remove service-level configs.

uninstall

- Purpose: remove installed prerequisites and transient host resources.
- Typical behavior: uninstall packages or services and clean transient state.

Roles & Responsibilities

The *automation developer* (or developer for short) designs, implements, and maintains automations and their representations. The developer keeps Git-managed playbooks timeless: they express the current desired state, not a history of migrations.

The *operator* runs published playbooks. The operator does not add one-time fixes to those playbooks. When a one-time change is required, the operator or developer runs an ephemeral automation from `tmp/`.

Statements

Representations

1. A developer ***must*** store each desired-state representation in a file.
2. A developer ***must not*** construct a desired-state representation only inside the automation tool.
3. A developer ***may*** instantiate a representation from a file (substitute environment-specific values) when the file is the source and the substitutions are not one-off literals.
4. A developer ***may*** keep orchestration-only values (connection parameters, lookup keys, create-versus-update control, and gathered facts) in the automation tool.
5. A developer ***may*** apply a trivial in-place mutation to an object the project does not own (for example, activate a product-provided record) without a representation file.

6. When the configuration object is source code, the source file ***is*** the representation. A developer ***may*** assemble record metadata around that source file in the automation tool.
7. A developer ***must*** store representation files outside the automation directory, in a directory intended for the system or subsystem being configured or modified.
8. A developer ***must not*** store artifacts in an automation directory.
9. A developer ***must*** place artifacts in a directory dedicated to artifacts for a service, or in a repository that provides a service for that type of artifact.
10. A developer ***may*** add a Markdown commentary file with the same basename as a representation file for notes that do not belong in the representation.
11. A developer ***must*** deploy onto a target host only those artifacts that host needs.

Variables

1. A developer ***must not*** introduce a named variable that is used only once to hold a literal.
2. A developer ***should*** name a value only when the automation uses that value in more than one place, or when an operator must override it per environment.
3. A developer ***should not*** extract a single play or task into a shared fragment unless reuse and maintenance value clearly outweigh the added indirection.

Ephemeral automations

1. A developer ***must not*** put a one-time update (for example, delete, rename, or deactivate an obsolete configuration) into a Git-managed playbook.
2. A developer ***must*** place an ephemeral automation in a `tmp/` directory under the automation directory.
3. A developer ***must*** list that `tmp/` directory in the project's Git ignore file.
4. A developer ***must not*** commit ephemeral automations to a Git repository.

Transient scripts

1. A developer ***should*** create a top-level `tmp` directory for each project.
2. A developer ***must*** add that top-level `tmp` directory to the Git ignore file.
3. A developer ***must*** create transient scripts in the top-level `tmp` directory.
4. A developer ***must not*** commit transient scripts to a Git repository.

Idempotency

1. A developer ***should*** make each automation idempotent: rerunning it must not have ill effects and must result in the same configuration state.
2. A developer ***should not*** execute a step unless the step is necessary. Each step should check whether the outcome is already in place and skip the step when it is. A developer ***may*** rely on a command or module that performs this check internally.

3. If an automation cannot test whether a step is necessary (for example, the API does not expose the current state), the automation ***must*** emit an informational message that the action is being redone before it re-executes the step.
4. When more than one automation platform can apply the same representation (for example, both Ansible and Terraform manage the same configuration object), a developer ***must*** make the platforms' results interchangeable: applying the representation with one platform and then with the other ***must*** produce no further changes from either platform.
5. A developer ***must not*** rely on a drift check that compares only added or changed fields. The check ***must*** detect a field's removal from the representation as a change, so that removing a property and reapplying actually clears it from the managed system.
6. For a representation managed by more than one platform, a developer ***must*** maintain a drift test that: applies the representation with the first platform; runs the second platform's dry-run (check) and confirms it reports no changes; applies with the second platform; and reruns the first platform and confirms it reports no changes. A developer ***should*** automate this test and run it whenever either platform's automation for that representation changes.

Terraform

1. A developer ***must*** configure Terraform to use a shared backend with state locking enabled.
2. When more than one team's Terraform configuration manages resources for the same application, and more than one of those configurations would otherwise declare the same resource, every team ***must*** manage that resource from one shared state.

Service-management playbooks

1. A developer ***must*** fully automate system configurations so that the project can always be built from source.
2. This project designates Ansible as the production automation platform. A developer ***must*** implement production automations on that platform, unless the automation manages or deploys a prerequisite of the platform.
3. A developer ***must*** follow the build, install, deploy, check, start, diagnose, test, stop, undeploy, and uninstall pattern of playbooks when managing a service.
4. For service management, a developer ***must*** name playbooks only with the names specified in the service-management pattern.
5. A developer ***must*** place each service-management playbook at the top level of the automation directory for that module.
6. A developer ***must*** name each service-management playbook with only the verb (for example `deploy.yml`, not `deploy_hadoop.yml` or `status.yml`).
7. For each role, a developer ***must not*** create more than one playbook that performs that service-management role.
8. A developer ***must*** implement playbooks to start and stop the service.
9. A developer ***should*** implement a diagnose playbook.

10. A developer ***should*** implement a check playbook for a module whose deploy playbook applies representation files.
11. A developer ***may*** omit playbooks from the service-management pattern when the way the service is built makes those verbs unnecessary.
12. A developer ***must not*** name a playbook `status.yml`. If a module already has `diagnose.yml`, the developer ***must*** merge any status checks into `diagnose.yml` and delete the status playbook.

Task playbooks

1. A developer ***must*** place each task playbook in a `tasks` subdirectory of the automation directory.
2. A developer ***must*** give each task playbook a descriptive kebab-case filename (for example `regenerate-k8s-certs.yml`).
3. A developer ***must not*** place a service-management playbook in `tasks/`.
4. A developer ***must not*** place a task playbook at the top level of the automation directory.
5. A task playbook ***must not*** change the semantic purpose of the calling service-management playbook.

Common directory

1. A developer ***may*** add a `common` subdirectory for variables shared by two or more playbooks in the module.
2. A developer ***must not*** store representations, artifacts, or task playbooks in `common/`.
3. A developer ***should*** omit `common/` when the module has no shared variables, or when those values live in representation files or project-level vars.

Diagnostic and test playbooks

1. For a service that has both test and diagnose playbooks, the diagnose playbook ***must*** include all checks in the test playbook for that service.
2. A diagnose playbook ***should*** include deeper and broader checks than a test playbook for the same service.
3. If a check wholly depends on the success of a prerequisite check, the dependent check ***must not*** run when the prerequisite check fails.
4. A developer ***should*** factor shared logic between test and diagnose playbooks into task playbooks in `tasks/` so the checks stay consistent.

Commentary

A representation file is the portable contract for a configuration object. Another team can apply the same document with a different automation tool or with a manual HTTP client. The automation tool loads the file and performs lookup, create, or update. It does not invent the document.

JSON is the usual interchange form when the managed system's API accepts JSON. YAML is acceptable when the target API accepts YAML. A commentary file holds rationale that the API payload cannot carry (for example, why a change-window filter uses Scheduled and Implement only).

A named variable that exists only to copy a literal into a representation adds a hop without adding an override or a reuse. Put the literal in the representation file.

Git-managed playbooks describe the timeless desired state. A leftover rename, a one-time delete, or a deactivate-the-old-name step is a migration. After it has run, it is noise and a source of accidental damage. Keep that work in `tmp/` and out of Git.

Check, diagnose, and test are three distinct read-only questions, not three names for the same check. Check answers "does what's deployed match the representation files" (configuration drift). Diagnose answers "is the service healthy" (operational status). Test answers "does the service behave correctly" (functional correctness). A module can fail one while passing the others: a service can be perfectly healthy and functionally correct while running a configuration that no longer matches its representation files (check fails, diagnose and test pass), or healthy with matching configuration but functionally broken (diagnose passes, test fails). On most automation platforms, check is close to free: a dry-run or plan mode the platform already provides (Terraform's `plan`, an rsync-based sync module's own dry-run/diff option) is often check's entire implementation, with no new assertions to write.

Service-management playbooks statement 2 names Ansible as this project's platform. The other statements do not depend on Ansible. They apply equally when a developer uses Terraform, Chef, or another tool.

Idempotency statements 4 through 6 cover a case plain idempotency (statements 1 through 3) does not: a representation that more than one platform can apply. A single platform can rerun its own upsert without ill effect and still drift relative to a second platform that manages the same object differently — for example, an Ansible task that merges new and changed fields into a stored value but never removes a field the representation dropped. The second platform's own idempotency check (comparing the whole value) then reports drift that the first platform's check missed. Making the two platforms' results interchangeable means an operator can apply with whichever platform is convenient, in either order, without the platforms fighting each other or masking a stale field. The drift test in statement 6 is how a developer proves that property, not just asserts it.

Terraform statement 2 is narrower than it may first read: it does not require every team working on the same application to merge into one state. It requires a single shared state only where a

resource would otherwise be declared by more than one team's configuration — for example, a provider resource that represents an entire shared collection as one value, with no resource type for a single member of that collection (a settings object holding a list every team appends entries to is one case). If two teams track that resource from separate states, each team's plan only knows about its own declared value, so each team's apply reverts or deletes the other team's entries. A shared backend (statement 1) does not by itself fix this: Terraform always refreshes against the real object before planning, and diffs the result against the configuration, not against another team's state. Fixing it takes one shared state tracking one composed configuration for that resource, not merely the same backend location. Partition state by component or team wherever no resource crosses that boundary; consolidate into one shared state only where a resource genuinely does.

An automation directory has at most ten service-management playbooks at its top level, named with the verbs in the table. Anything those playbooks call that is itself a playbook lives in `tasks/` under a descriptive kebab-case name. `common/` is only for shared variables. Representations and artifacts do not belong in either subdirectory; they live outside the automation directory. After representations moved out of the Ansible tree, many modules no longer need `common/` at all.

References

1. Key Words for Authoring Technical Standards
2. Technical Policies and Guidelines for Technical Policies and Guidelines
3. Development (project-level `tmp/` for transient documentation)